

Sri Lanka Institute of Information Technology



DESIGN PATTERN IMPLEMENTATION REPORT

Web-Based Library Management System

SE2030 - Software Engineering

Group ID: 2026-Y2-S1-MLB-B10G2-09

Final Presentation Preparation - October 2026

Implemented Pattern: Observer Design Pattern

Student ID	Name	Assigned Major Function
IT25101473	Ahamed M.S.S.	User Authentication and User Management
IT25102278	Koswatta A.C.W.W.M.R.	Borrowing and Issuing Management
IT25103242	Andrewdilan T.	Return and Fine Management
IT25100725	Fernando K.M.D.	Administration, Reports, and Notification Management
IT25102701	Liyanaarachchi G. C. H.	Book Search and Reservation Management
IT25100139	Harikesh S.	Book and Category Management

1. Implementation Summary

The final SmartLibrary Connect project now contains a real Observer Pattern implementation integrated into the normal application workflow. The pattern is used for event-driven library notifications rather than as a separate demonstration-only program. Reservation, borrowing, return/fine and scheduled reminder operations publish events through a common Subject interface. Multiple Observer implementations react to the same event without the business service needing to know the details of each reaction.

No new external library or database schema change was required. The existing Spring Boot, JPA, SQL Server, frontend notification view and administrator activity-log view are reused.

1.1 Why Observer Fits This System

- One library action can require more than one reaction, such as a member notification and an administrator audit record.
- Reservation, borrowing and fine services remain focused on their own business rules instead of directly implementing every notification channel.
- A future channel such as SMS, push notification or another audit target can be added as another observer without rewriting the publishing services.
- The pattern directly supports the documented due-date reminders, overdue alerts, reservation notices and borrowing/fine notifications.

1.2 Pattern Components in the Code

Observer Role	Project Class	Responsibility
Event data	LibraryEvent	Carries user, message, notification type, source function and event name.
Subject interface	LibraryEventSubject	Defines addObserver, removeObserver and notifyObservers.
Concrete Subject	LibraryNotificationSubject	Maintains the observer list and broadcasts each event.
Observer interface	LibraryObserver	Defines update(LibraryEvent).
Concrete Observer 1	InAppNotificationObserver	Creates the member notification through NotificationService and existing email stub.
Concrete Observer 2	NotificationActivityLogObserver	Creates the administrator activity-log/audit entry.

2. Observer Pattern Architecture

The diagram below shows the implementation as it now exists in the project. Four existing application areas publish library events. The Concrete Subject broadcasts each event to two Concrete Observers, producing both a member-facing notification and an administrator-facing audit record.

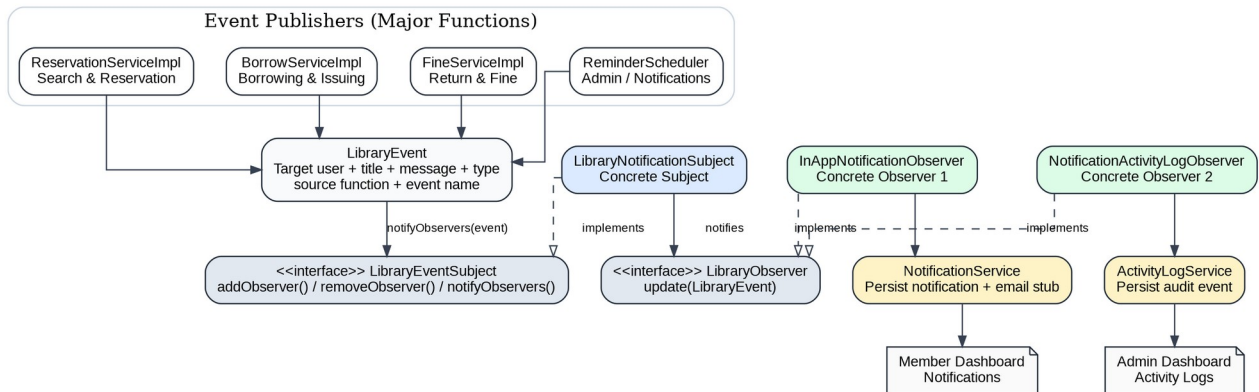


Figure 1. Observer Pattern integrated into SmartLibrary Connect

2.1 Runtime Flow

- A normal library action occurs, for example a member creates a reservation.
- The responsible service creates a LibraryEvent and calls LibraryEventSubject.notifyObservers(event).
- LibraryNotificationSubject broadcasts the event to all registered LibraryObserver implementations.
- InAppNotificationObserver stores the member notification and runs the existing email-notification stub.
- NotificationActivityLogObserver records an OBSERVER_NOTIFICATION entry for administrator monitoring.
- The original library operation continues without containing notification-channel logic.

3. Where the Pattern Is Applied

The Observer Pattern is a group-level design improvement, but its direct integration points belong to specific major functions. The table distinguishes direct code integration from modules that remain unchanged.

Member	Assigned Function	Observer Integration	Main Code Location
Liyanaarachchi G. C. H. IT25102701	Book Search and Reservation Management	DIRECT - publishes reservation created/status/cancelled events.	ReservationServiceImpl.java
Koswatta A.C.W.W.M.R. IT25102278	Borrowing and Issuing Management	DIRECT - publishes issue, renewal and return events.	BorrowServiceImpl.java
Andrewdilan T. IT25103242	Return and Fine Management	DIRECT - publishes fine-created and fine-status events.	FineServiceImpl.java
Fernando K.M.D. IT25100725	Administration, Reports, and Notification Management	DIRECT / CORE - reminder publisher plus Subject and Concrete Observers for notifications and audit logs.	ReminderScheduler.java + designpattern/observer package
Ahamed M.S.S. IT25101473	User Authentication and User Management	INDIRECT - no pattern code changed; existing User ID identifies the event target.	No direct change
Harikesh S. IT25100139	Book and Category Management	INDIRECT - no pattern code changed; existing Book data is used in event messages.	No direct change

3.1 Events Published

The following events are now handled through the same Observer infrastructure: RESERVATION_CREATED, RESERVATION_STATUS_CHANGED, RESERVATION_CANCELLED, BOOK_ISSUED, BORROW_RENEWED, BOOK_RETURNED, FINE_CREATED, FINE_STATUS_CHANGED, DUE_DATE_REMINDER and OVERDUE_ALERT.

4. Implementation Evidence

The following shortened excerpts are based on the final project source. They show the required Subject/Observer structure and a real integration point in the application.

4.1 Subject and Observer Interfaces

```
public interface LibraryEventSubject {
    void addObserver(LibraryObserver observer);
    void removeObserver(LibraryObserver observer);
    void notifyObservers(LibraryEvent event);
}

public interface LibraryObserver {
    void update(LibraryEvent event);
}
```

4.2 Concrete Subject Broadcast

```
@Component
public class LibraryNotificationSubject implements LibraryEventSubject {
    private final List<LibraryObserver> observers = new CopyOnWriteArrayList<>();

    public LibraryNotificationSubject(List<LibraryObserver> discoveredObservers) {
        observers.addAll(discoveredObservers);
    }

    public void notifyObservers(LibraryEvent event) {
        for (LibraryObserver observer : observers) {
            observer.update(event);
        }
    }
}
```

4.3 Example Publisher - Reservation Function

```
eventSubject.notifyObservers(new LibraryEvent(
    memberId,
    "Reservation created",
    "Your reservation for '" + book.getTitle() + "' is active.",
    NotificationType.RESERVATION,
    "Book Search and Reservation Management",
    "RESERVATION_CREATED"
));
```

4.4 Concrete Observers

```
// Member-facing notification observer
notificationService.send(event.userId(), event.title(),
    event.message(), event.notificationType());

// Administrator audit observer
activityLogService.log("SYSTEM-OBSERVER",
    "OBSERVER_NOTIFICATION", details);
```

5. Final Presentation Demonstration

The pattern can be demonstrated using the normal application UI; no special demo-only screen is required.

1. Run the project on the Windows laptop exactly as before: start SQL Server, use JDK 17, and run SmartLibraryApplication.java or RUN_LIBRARY.bat.
2. Log in as a Library Member and reserve an available book.
3. Open Member Dashboard -> Notifications and show the new reservation notification.
4. Log in as Administrator and open Activity Logs. Show the matching OBSERVER_NOTIFICATION record created from the same event.
5. Log in as Librarian and issue or renew a book for the member; then show the resulting member notification and admin audit entry.
6. For the return/fine function, return an overdue book and show the fine notification. Updating the fine status also publishes an Observer event.
7. Point to the IntelliJ console line beginning with [OBSERVER] to explain that one event is being broadcast to multiple observers.

5.1 Suggested Viva Explanation

"We used the Observer Pattern because a single library event can require multiple independent reactions. The service publishes one LibraryEvent. LibraryNotificationSubject is the Concrete Subject and automatically notifies all LibraryObserver implementations. InAppNotificationObserver creates the member notification, while NotificationActivityLogObserver creates an administrator audit entry. This keeps reservation, borrowing and fine logic loosely coupled from the notification channels and makes it easy to add another observer later."

5.2 Windows Files Added for Verification

- TEST_OBSERVER_PATTERN.bat - runs the focused Observer unit test.
- FINAL_CHECK.bat - compiles the complete project and runs tests with Gradle.
- DESIGN_PATTERN_IMPLEMENTATION.md - detailed class/event/member mapping included in the project ZIP.

6. Verification and Final Notes

The design-pattern package and every affected service class were syntax/type checked against the project interfaces and entities using Java compilation with local framework stubs. The modified subset compiled successfully. A standalone Observer smoke test also passed, confirming that one event is delivered to two registered observers.

The full Gradle/Spring Boot build could not be executed inside the preparation environment because the Gradle wrapper attempted to download its distribution and this environment has no external network access. The supplied Windows project therefore includes FINAL_CHECK.bat so the team can perform the complete Gradle compile/test check on the presentation laptop before the final demonstration.

6.1 Final Checklist Before Presentation

- Run FINAL_CHECK.bat once while the laptop has the required Gradle dependencies available.
- Start SQL Server and confirm the existing database connection in application.properties.
- Run the application and complete one reservation, one borrowing/renewal and one return/fine flow.
- Confirm Member Dashboard -> Notifications shows the generated messages.
- Confirm Admin Dashboard -> Activity Logs shows OBSERVER_NOTIFICATION records.
- Make sure the member responsible for each directly affected function understands where that function publishes its LibraryEvent.
- Keep DESIGN_PATTERN_IMPLEMENTATION.md open as a quick viva reference.

6.2 Result

The Observer Pattern is now integrated into the actual Web-Based Library Management System rather than remaining only as a design recommendation. It directly supports four major functional areas while preserving the existing Windows run workflow and without requiring a new external dependency or database table.